BICA Reborn: An Annotation-Based Session Type Checker for Java Objects with Parallel Composition

Technical Report — Phases 1–7: Complete Pipeline

Alexandre Zua Caldeira
LASIGE, Faculdade de Ciências, Universidade de Lisboa
Lisbon, Portugal
azcaldeira@fc.ul.pt

Vasco T. Vasconcelos
LASIGE, Faculdade de Ciências, Universidade de Lisboa
Lisbon, Portugal
vmvasconcelos@fc.ul.pt

Abstract

We present Bica Reborn, a Java annotation-based session type checker for objects that extends classical session types with a *parallel composition* constructor $S_1 \parallel S_2$. The parallel constructor enables specifying concurrent access patterns on shared objects and forces the session type state space to form a *lattice*—the algebraic structure central to the thesis "Session Types as Algebraic Reticulates." Bica Reborn is implemented in Java 21 across seven completed phases: (1) an immutable AST with sealed interfaces and records, tokenizer, recursive-descent parser, and round-trip–preserving pretty-printer; (2) state-space construction including product lattice construction for the $\parallel$ constructor; (3) lattice verification via SCC-based quotient construction, together with concurrency annotations (`@Session`, `@Shared`, `@ReadOnly`, `@Exclusive`) and a Java annotation processor for compile-time verification; (4) termination checking, well-formedness analysis for parallel composition (WF-Par), and a morphism hierarchy (isomorphism, embedding, projection, Galois connection) between state spaces; (5) a standalone CLI tool for session type analysis; (6) typestate checking that verifies client code follows session protocols; (7) automatic JUnit 5 test generation from session type state spaces, producing protocol conformance tests for valid paths, violations, and incomplete protocols. The implementation spans 12 packages, is validated by 871 unit tests including 30 benchmark protocols from real-world systems (SMTP, OAuth 2.0, WebSocket, MCP, A2A, Raft, Saga, Kafka) and the session-type literature, and achieves full pipeline parity with the Reticulate Python reference implementation. The test generator produces 4,552 conformance tests from the 30 benchmarks—an average of 152 tests per protocol.

CCS Concepts: • **Software and its engineering → Formal software verification**; *Formal methods*; *Software verification and validation.*

Keywords: session types, parallel composition, lattice theory, Java annotation processing, protocol verification

1 Introduction

Session types [7] discipline communication by assigning types to interaction protocols: a session type prescribes the legal sequences of method calls, branches, and selections that may occur on an object interface. Since their introduction, session types have been applied to objects [6, 12], multiparty protocols [8], and practical toolchains such as Mungo [9] and Scribble [13].

A fundamental limitation of existing session type systems for objects is the assumption of *linear ownership*: exactly one client holds an object reference at any time. This precludes modeling *concurrent access patterns*—situations where multiple threads, tasks, or agents interact with the same object simultaneously.

Bica Reborn addresses this limitation by introducing a *parallel composition* constructor $S_1 \parallel S_2$, which specifies that an object may be shared between two concurrent execution paths, each following its own sub-protocol. This constructor is the key novelty of the tool and—as established in the companion formal specification [2]—it is precisely what forces the session type state space to form a *lattice*.

Bica Reborn is the successor to the original Bica tool [6] and a key component of the PhD thesis "Session Types as Algebraic Reticulates." It reimplements and extends the algorithms from the Reticulate Python library, which serves as the verified reference implementation with 642 tests across 8 modules.

Contributions of this report.

- An immutable, exhaustive AST representation for session types with parallel composition, using Java 21 sealed interfaces and records (§3).
- A tokenizer, recursive-descent parser, and round-trip–preserving pretty-printer with Unicode support (§4–§6).
- State-space construction from session type ASTs, including product lattice construction $L(S_1) \times L(S_2)$ for parallel composition.

- Lattice verification via SCC-based quotient construction using Tarjan's algorithm, with pairwise meet/join computation on the quotient DAG.
- Termination checking and well-formedness analysis for parallel composition (WF-Par): termination, guardedness, variable scoping, and no nested ∥.
- A concurrency level lattice (Shared > Sync, Read-Only > Exclusive) for thread safety analysis of concurrent method pairs.
- A morphism hierarchy between state spaces: isomorphism, order embedding, projection, and Galois connection (§7).
- A Java annotation processor for compile-time verification of `@Session`, `@Shared`, `@ReadOnly`, and `@Exclusive` annotations.
- A standalone CLI tool for session type analysis with text, DOT, and test generation output modes.
- Typestate checking: a pure state-machine simulator that verifies method call sequences against session protocols, integrated with the annotation processor for compile-time verification (§8).
- Automatic JUnit 5 test generation from session type state spaces, producing three categories of tests—valid paths, violations, and incomplete protocols—with bounded path enumeration for recursive types (§9).
- Validation against 871 unit tests across 19 test classes, including 30 benchmark protocols from real-world systems (§10).

Scope. This report covers all seven phases of the implementation, from parsing through test generation. The complete pipeline transforms a `@Session` annotation string through parsing, state-space construction, lattice checking, termination analysis, thread safety verification, morphism classification, typestate checking, and protocol conformance test generation.

2 Session Type Grammar

Bica Reborn parses session types from `@Session` annotation strings. The grammar extends standard session type syntax with the parallel constructor and sequencing sugar:

Definition 2.1 (Session type syntax)**.**

S ::=	$\&\{m_1 : S_1, \ldots, m_n : S_n\}$	branch (external choice)
|	$\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$	selection (internal choice)
|	$S_1 \parallel S_2$	parallel composition
|	$\mu X.\, S$	recursion
|	X	type variable
|	end	terminated
|	$S_1 \,.\, S_2$	sequencing (sugar)

Desugaring. When the left-hand side of sequencing is a bare identifier m, the parser desugars it into a single-method branch: $m \,.\, S$ becomes $\&\{m : S\}$. This models a mandatory method call followed by a continuation—the most common pattern in object protocols.

Concrete syntax. The annotation string syntax uses ASCII-friendly notation: `&{...}` for branch, `⊕{...}` for selection, `(S1 | S2)` or `‖{S1, S2}` for parallel, and `rec X . S` for recursion. The tokenizer also accepts Unicode alternatives: ⊕ for +, ‖ for ||, and μ for rec.

Example. A shared file object with concurrent read/write access:

```
@Session("init . &{open: +{OK: (read . end || write
      . end)"
       + " . close . end, ERROR: end}}")
public class SharedFile { ... }
```

After `open()`, the object enters a parallel region where a reader and writer access it concurrently. Both must reach `end` before `close()` becomes available.

3 Abstract Syntax Tree

The AST exploits three Java 21 features: *sealed interfaces* for closed type hierarchies, *records* for immutable value types with structural equality, and *pattern matching* for exhaustive dispatch.

3.1 Sealed Interface

The root of the AST hierarchy is a sealed interface that permits exactly seven implementations:

```
public sealed interface SessionType
    permits End, Var, Branch, Select,
            Parallel, Rec, Sequence { }
```

The **`sealed`** modifier guarantees that no other class can implement `SessionType`. The Java compiler can therefore verify that **`switch`** expressions over `SessionType` are exhaustive, eliminating "forgot to handle this case" errors at compile time.

3.2 Record Implementations

Each AST node is a record, providing immutability, structural `equals()`/`hashCode()`, and null rejection via compact constructors. Table 1 summarises the seven node types.

Shared* `Choice` *record. Both `Branch` and `Select` contain ordered lists of labelled choices. Rather than duplicating the type, `Branch.Choice` is reused by both. The choice list is defensively copied via `List.copyOf()` in the record's compact constructor:

```
public record Branch(List<Choice> choices)
        implements SessionType {
    public record Choice(String label,
                         SessionType body) {
        public Choice {
```

Table 1. AST node types. All are records implementing `SessionType`.

Record	Components	Description
`End`	—	Terminated session
`Var`	`String name`	Type variable reference
`Branch`	`List<Choice> choices`	External choice $\&\{\ldots\}$
`Select`	`List<Choice> choices`	Internal choice $\oplus\{\ldots\}$
`Parallel`	`left, right`	Parallel $S_1 \parallel S_2$
`Rec`	`String var, body`	Recursion $\mu X.\, S$
`Sequence`	`left, right`	Sequential $S_1 \,.\, S_2$
`Choice`	`String label, body`	Labelled choice (inner record of `Branch`)

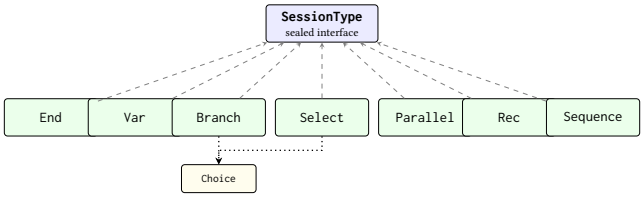

Figure 1. AST sealed interface hierarchy. Solid boxes are records; dashed arrows indicate `implements`. `Choice` is an inner record of `Branch`, reused by `Select` (dotted arrows).

```
            Objects.requireNonNull(label);
            Objects.requireNonNull(body);
        }
    }
    public Branch {
        Objects.requireNonNull(choices);
        choices = List.copyOf(choices);
    }
}
```

This guarantees structural immutability: the list cannot be modified after construction, and any attempt to call `add()` on the returned list throws `UnsupportedOperationException`.

Figure 1 shows the AST hierarchy.

4 Tokenizer

The tokenizer performs a single left-to-right pass over the source string, producing an immutable list of `Token` records plus a trailing EOF sentinel. Table 2 lists all twelve token kinds.

Two-character lookahead. The `||` token requires one character of lookahead. A single `|` is not a valid token, so there is no ambiguity.

Position tracking. Every token records its character offset in the source string. This enables precise error diagnostics:

```
at position 14: expected COLON, got IDENT
('end')
```

Table 2. Token kinds.

Kind	Symbol(s)	Notes
LBRACE/RBRACE	`{ }`	Choice delimiters
LPAREN/RPAREN	`( )`	Grouping / parallel
AMPERSAND	`&`	Branch introducer
PLUS	`+` or $\oplus$	Selection introducer
COLON	`:`	Label separator
COMMA	`,`	Choice separator
DOT	`.`	Sequencing operator
PAR	`||` or $\parallel$	Parallel operator
IDENT	`[A-Za-z_]...`	Identifiers & keywords
EOF	—	End-of-input sentinel

Table 3. Parser methods and their grammar rules.

Method	Rule	Assoc.
`seqExpr`	*atom* (. *seqExpr*)?	Right
`atom`	*branch* | *select* | *paren* | ...	—
`parseBranch`	&{ *choiceList* }	—
`parseSelect`	+{ *choiceList* }	—
`parenOrParallel`	(S (|| S)?)	—
`braceParallel`	||{ S , S }	—
`parseRec`	rec X . S	—
`choiceList`	m:S (, m:S)*	—

Unicode support. The tokenizer recognises three Unicode alternatives: $\oplus$ (U+2295) for `+`, $\parallel$ (U+2225) for `||`, and μ (U+03BC) for `rec`. This supports direct transcription from formal notation into annotation strings.

5 Parser

The parser is a hand-written recursive-descent parser [1] that transforms a token list into a `SessionType` AST. Table 3 maps grammar rules to parser methods.

5.1 Desugaring

After parsing *left* . *right*, the parser checks whether *left* is a `Var` node using Java 21 pattern matching:

```
if (left instanceof Var v) {
    return new Branch(List.of(
        new Choice(v.name(), right)));
}
return new Sequence(left, right);
```

When *left* is a bare identifier, it is reinterpreted as a method name and the expression desugars to a single-method `Branch`. Otherwise, a `Sequence` node is produced.

5.2 Right-Associativity

The sequencing operator is right-associative: $a \,.\, b \,.\, c \,.\, \mathrm{end}$ parses as $a \,.\, (b \,.\, (c \,.\, \mathrm{end}))$, producing nested single-method branches $\&\{a : \&\{b : \&\{c : \mathrm{end}\}\}\}$.

5.3 Error Reporting

The parser does not attempt error recovery. On the first syntax error, a `ParseError` is thrown with the exact source position and a message identifying the expected and actual token kinds. The keywords `rec` and `end` are rejected as recursion variable names.

6 Pretty-Printer

The pretty-printer renders an AST back to a human-readable string using a pattern matching **`switch`** over the sealed `SessionType` interface:

```
public static String pretty(SessionType node) {
  return switch (node) {
    case End e    -> "end";
    case Var v    -> v.name();
    case Branch b when b.choices().size() == 1
      -> { Choice c = b.choices().getFirst();
           yield c.label()+" . "+pretty(c.body());}
    case Branch b -> "&{"+fmtChoices(b)+"}";
    case Select s -> "+{"+fmtChoices(s)+"}";
    case Parallel p-> "("+pretty(p.left())
                     +" || "+pretty(p.right())+")";
    case Rec r    -> "rec "+r.var()
                     +" . "+pretty(r.body());
    case Sequence q-> pretty(q.left())
                     +" . "+pretty(q.right());
  };
}
```

Sugar preservation. Single-method `Branch` nodes are printed as sequencing sugar ($m\,.\,S$) via the guarded pattern **`case`** `Branch b when b.choices().size() == 1`. This ensures the *round-trip property*:

Proposition 6.1 (Round-trip). *For all valid session type strings s:*

$$parse(pretty(parse(s))) = parse(s)$$

This property is verified by test for all 30 benchmark protocols (Section 10).

7 Formal Foundations

We briefly state the theoretical results that motivate the Bica Reborn architecture. Full proofs appear in the companion specification [2].

7.1 Product Lattice from Parallel Composition

Theorem 7.1 (Lattice structure). *If $L_1 = L(S_1)$ and $L_2 = L(S_2)$ are lattices, then $L(S_1 \parallel S_2) = L_1 \times L_2$, ordered componentwise, is a lattice.*

The product poset $L_1 \times L_2$ has:

- **Top**: $(\top_1, \top_2)$—the fork point.
- **Bottom**: $(\bot_1, \bot_2) = (\text{end}, \text{end})$—the join point.
- **Meet**: $(s_1 \wedge_1 s_1',\ s_2 \wedge_2 s_2')$, componentwise.
- **Join**: $(s_1 \vee_1 s_1',\ s_2 \vee_2 s_2')$, componentwise.

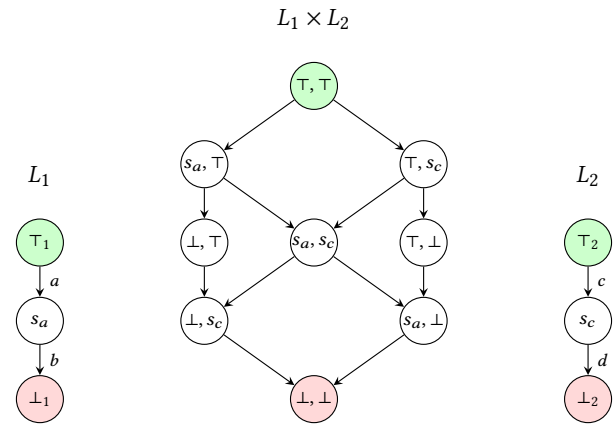

Figure 2. Product lattice $L_1 \times L_2$ for $a.b.\text{end} \parallel c.d.\text{end}$, with the two factor lattices L_1 (left) and L_2 (right).

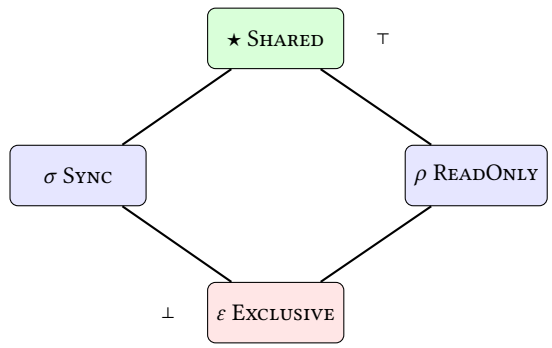

Figure 3. Concurrency level lattice. Two methods in a $\parallel$ branch are compatible iff $l_1 \wedge l_2 \neq \varepsilon$.

Table 4. Concurrent compatibility matrix. ✓ = safe, × = data race.

	ε	ρ	σ	$\star$
ε Exclusive	×	×	×	×
ρ ReadOnly	×	✓	✓	✓
σ Sync	×	✓	✓	✓
$\star$ Shared	×	✓	✓	✓

Figure 2 illustrates the 3×3 product lattice for $a\,.\,b\,.\,\text{end} \parallel c\,.\,d\,.\,\text{end}$.

7.2 Thread Safety as a Lattice Property

Bica Reborn classifies each method into a concurrency level drawn from the four-element lattice shown in Figure 3: exclusive (ε), readonly (ρ), sync (σ), and shared ($\star$).

Two methods are *concurrently compatible* iff their meet is not ε:

$$compatible(l_1, l_2) \iff l_1 \wedge l_2 \neq \varepsilon$$

Table 4 gives the full compatibility matrix.

7.3 Well-Formedness of Parallel Composition

The well-formedness rule for $S_1 \parallel S_2$ requires six conditions:

1. Both S_1 and S_2 are well-formed.
2. **Termination:** every recursive definition $\mu X.\, S$ reachable inside S_1 or S_2 has an exit path to end.
3. **Guardedness:** all recursion variables are preceded by observable actions (&, ⊕, or method calls).
4. **Variable scoping:** no cross-branch recursion variables.
5. **No nested ||:** neither S_1 nor S_2 contains a || constructor.
6. **Thread safety:** all method pairs across branches are concurrently compatible.

These conditions are all checked statically by the tool.

8 Typestate Checking

Phase 6 adds *typestate checking*: a pure state-machine simulator that verifies whether a sequence of method calls on a `@Session`-annotated object follows the session protocol.

8.1 Checker Design

`TypestateChecker.check()` takes a `StateSpace`, a list of `MethodCall` records, an initial state map, and a flag indicating whether to verify that all tracked variables reach the bottom (end) state. The algorithm:

1. Iterates through method calls in source order.
2. For each call on a tracked variable, checks whether the method is enabled at the current state via `ss.enabledLabels(state)`.
3. If enabled, transitions to the target state via `ss.step(state, method)`.
4. If disabled, records a `TypestateError` with the current state, the disabled method, and the set of enabled methods.
5. At the end, if end-checking is enabled, verifies that all tracked variables are in the bottom state; any variable not at bottom generates an incomplete-protocol error.

The checker does not fail-fast: it collects all violations, enabling comprehensive error reporting in a single pass.

8.2 Compiler Integration

`MethodCallExtractor` bridges the checker to the Java compiler via the `com.sun.source` API. It walks method bodies using a `TreePathScanner`, tracking local variables whose type is a `@Session`-annotated class. Calls on tracked variables are extracted as `MethodCall` records with source positions for precise error diagnostics.

The annotation processor (`SessionProcessor`) invokes the extractor and checker at compile time: for each method in a client class, it extracts the call sequence, builds the state space from the `@Session` annotation, and reports violations as compiler diagnostics.

9 Test Generation

Phase 7 introduces *automatic test generation* from session type state spaces. Given a `@Session` annotation, Bica Reborn generates a complete JUnit 5 test class with three categories of tests:

1. **Valid paths** — bounded DFS from top to bottom, producing one test per distinct top-to-bottom trace.
2. **Violation points** — BFS-discovered shortest paths to each reachable state, combined with each disabled method at that state.
3. **Incomplete protocols** — proper prefixes of valid paths where the final state is not bottom, highlighting that enabled methods remain.

9.1 Path Enumeration

Valid paths are enumerated by bounded depth-first search. To handle recursive protocols, the `maxRevisits` parameter bounds how many times a state may be revisited within a single path:

```
dfs(state, path, visitCounts):
  if state == bottom: emit(path)
  if visitCounts[state] > maxRevisits: return
  visitCounts[state]++
  for (label, target) in ss.enabled(state):
    dfs(target, path + (label, target))
  visitCounts[state]--
```

For the recursive type $\mu X.\, \&\{next : X,\ done : \mathrm{end}\}$ with `maxRevisits` = 2, this produces three paths: $[done]$, $[next, done]$, and $[next, next, done]$.

A `maxPaths` cap (default 100) truncates enumeration for protocols with large or infinite path spaces, emitting a warning comment in the generated source.

9.2 Generated Output

The generator produces a compilable JUnit 5 class:

```
class FileHandleProtocolTest {
  // ===== Valid paths (2) =====
  @Test
  void validPath_open_read_close() {
    FileHandle obj = new FileHandle();
    obj.open(); obj.read(); obj.close();
  }
  // ===== Violations (3) =====
  @Disabled("Protocol violation: 'read'"
    + " not enabled in initial state")
  @Test
  void violation_initial_read() {
    FileHandle obj = new FileHandle();
    obj.read(); // VIOLATION
  }
  // ===== Incomplete protocols (2) =====
  @Disabled("Incomplete: object not in"
    + " terminal state after [open]")
  @Test
```

```
  void incomplete_open() {
    FileHandle obj = new FileHandle();
    obj.open();
    // NOT COMPLETE: [read, write] remain
  }
}
```

Violation tests support two styles: `DISABLED_ANNOTATION` (default) marks them `@Disabled` for manual completion, while `CALL_ANYWAY` wraps the violation in `assertThrows(IllegalStateException.class, ...)`.

9.3 CLI Integration

The CLI exposes test generation via:

```
bica -test-gen -class-name FileHandle
"open . &{read: close.end, write: close.end}"
```

The flags `-test-gen` and `-dot` are mutually exclusive. `-class-name` is required with `-test-gen`. Additional options: `-package`, `-var-name`, `-max-revisits`.

9.4 Benchmark Coverage

Table 5 shows the test generation results across all 30 benchmarks with `maxRevisits` = 2 and `maxPaths` = 100.

The generator produces **4,552 tests** across the 30 benchmarks—an average of 152 tests per protocol. Six benchmarks (20%) hit the 100-path truncation cap, all of which use parallel composition or deep recursion. Violations dominate for protocols with many states and few enabled methods per state (e.g., Saga Orchestrator: 174 violations), while incomplete prefixes are most numerous for protocols with parallel composition (e.g., Kafka Consumer: 253 incomplete prefixes).

Table 5. Test generation coverage across 30 benchmarks. Paths = valid top-to-bottom traces, Viol. = disabled-method violations, Inc. = incomplete protocol prefixes, † = truncated at 100 paths.

#	Protocol	Paths	Viol.	Inc.
1	Java Iterator	3	8	7
2	File Object	3	15	9
3	SMTP	7	40	17
4	HTTP Connection	13	21	21
5	OAuth 2.0	16	55	17
6	Two-Buyer	7	49	12
7†	MCP	100	31	140
8	A2A	9	21	9
9	File Channel	43	36	92
10	ATM	22	49	29
11	Reentrant Lock	7	23	18
12†	WebSocket	100	30	112
13	DB Transaction	42	23	31
14	Pub/Sub	21	14	26
15	DNS Resolver	42	16	62
16	Reticulate Pipeline	6	112	18
17	GitHub CI Workflow	6	87	13
18	TLS Handshake	4	81	11
19	Raft Leader	79	44	169
20†	MQTT Client	100	50	149
21	Circuit Breaker	62	35	123
22†	Conn. Pool	100	79	202
23	gRPC BiDi	42	11	49
24	Blockchain Tx	9	32	8
25†	Kafka Consumer	100	67	253
26	Rate Limiter	14	12	20
27	Saga Orchestrator	92	174	175
28	Two-Phase Commit	3	34	5
29†	Leader Repl.	100	98	182
30	Failover	14	32	28
	Total	**1,166**	**1,379**	**2,007**

10 Evaluation

10.1 Test Suite

The implementation is validated by 871 JUnit 5 tests across 19 test classes spanning all seven phases. Table 6 summarises the test distribution.

All 871 tests pass. Build time is approximately 15 seconds on a standard development machine (including Maven dependency resolution and compilation).

10.2 Benchmark Protocols

The full pipeline is validated against 30 protocols from real-world systems and the session-type literature. Table 7 lists all protocols with their state-space metrics.

Of the 30 benchmarks, 12 (40%) use the parallel constructor, which is the key novelty of Bica Reborn. The largest is the Saga Orchestrator (20 states, 37 transitions), combining parallel composition with sequencing and selection. All 30 protocols are confirmed to form bounded lattices, and both the Java and Python implementations agree on all metrics. Each protocol is also tested for parse success and round-trip fidelity (Proposition 6.1).

10.3 Code Metrics

Table 8 summarises the code size. The production code has **zero external dependencies**—only the Java 21 standard library.

10.4 Comparison with Python Reference

Table 9 compares the Java and Python implementations across the full pipeline. Both tools now have complete pipeline parity: parsing, state-space construction, lattice checking, termination analysis, morphism hierarchy, and visualization/CLI.

The Java implementation is larger due to the language's more verbose syntax (package declarations, imports, explicit type annotations) and the additional annotation processor

Table 6. Test distribution by class and category.

Test class	Tests	Phase	Coverage area
`SessionTypeTest`	23	1	Equality, immutability, null rejection
`TokenizerTest`	9	1	Tokens, positions, Unicode, errors
`ParserTest`	75	1	Constructs, sequencing, benchmarks
`PrettyPrinterTest`	11	1	Node types, round-trip
`StateSpaceTest`	120	2	State space + product construction
`LatticeCheckerTest`	100	2	Lattice properties, SCC quotients
`TerminationCheckerTest`	80	2	Termination, WF-Par conditions
`ConcurrencyLatticeTest`	40	3	Concurrency level ordering
`MethodClassifierTest`	25	3	Method classification
`ThreadSafetyCheckerTest`	70	3	Thread safety analysis
`ThreadSafetyResultTest`	20	3	Result records, violations
`SessionProcessorTest`	50	3	Annotation processor pipeline
`MorphismCheckerTest`	80	4	Iso, embedding, projection, Galois
`BicaCliTest`	92	5	CLI text/DOT/test-gen, error handling
`TypestateCheckerTest`	40	6	Linear, branch, recursive protocols
`TypestateIntegrationTest`	15	6	Compiler integration, diagnostics
`PathEnumeratorTest`	30	7	DFS paths, BFS violations, prefixes
`TestGeneratorTest`	42	7	Output format, config, end-to-end
`BenchmarkCoverageTest`	1	7	Coverage stats across 30 benchmarks
Total	**871**		

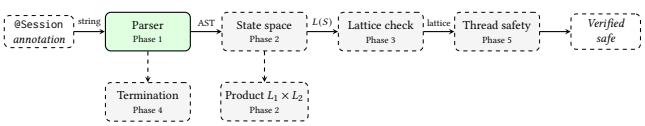

Figure 4. Bica Reborn pipeline. All seven phases are complete. The pipeline transforms a `@Session` annotation string through parsing, state-space construction, lattice checking, thread safety verification, typestate checking, and test generation.

component, but the algorithmic logic is a direct port. Java provides stronger compile-time guarantees: sealed interfaces ensure exhaustive dispatch (the Python match statement does not enforce exhaustiveness), and `Objects.requireNonNull` catches null errors at construction time. The annotation processor is a Java-specific contribution with no Python counterpart, enabling compile-time verification of session type protocols in standard Java build pipelines.

11 System Architecture

Figure 4 shows the pipeline. All seven phases are implemented.

The implementation is organised into 12 packages, each corresponding to a well-defined concern. Table 10 lists all packages.

Phase 1: AST and Parser (118 tests). The sealed `SessionType` interface with 7 record implementations provides an immutable, exhaustive AST (§3). The tokenizer supports Unicode alternatives ($\oplus$, $\|$, μ) and position-aware error reporting (§4). The recursive-descent parser desugars $m\ .\ S$ into single-method branches (§5), and the pretty-printer preserves the round-trip property (§6).

Phase 2: State-space construction (170 new tests, 288 total). `StateSpaceBuilder` constructs the state-space graph $L(S)$ from the AST. Recursion is handled via the placeholder-merge pattern: allocate a temporary state identifier for the variable, build the body, then redirect transitions. `ProductStateSpace`

Table 7. Benchmark results for 30 session type protocols. $|Q|$ = states, $|\delta|$ = transitions, $|SCC|$ = SCCs after quotient, $\|$ indicates use of parallel composition. All 30 form bounded lattices.

#	Protocol	$\|Q\|$	$\|\delta\|$	$\|SCC\|$	$\|$	Domain
1	Java Iterator	4	4	2		Pattern
2	File Object	5	5	4		I/O
3	SMTP	7	8	4		Network
4	HTTP Connection	5	7	3		Network
5	OAuth 2.0	7	11	6		Security
6	Two-Buyer	10	14	10	✓	Commerce
7	MCP	7	17	5	✓	AI Agent
8	A2A	5	7	4		AI Agent
9	File Channel	7	12	7	✓	I/O
10	ATM	7	11	6		Finance
11	Reentrant Lock	6	7	2		Concurrency
12	WebSocket	6	15	6	✓	Network
13	DB Transaction	5	9	3		Database
14	Pub/Sub	4	7	3		Messaging
15	DNS Resolver	4	8	2		Network
16	Reticulate Pipeline	14	18	14	✓	Tooling
17	GitHub CI Workflow	11	13	11	✓	CI/CD
18	TLS Handshake	13	15	13		Security
19	Raft Leader	6	11	2		Distributed
20	MQTT Client	8	20	6	✓	IoT
21	Circuit Breaker	6	10	2		Resilience
22	Connection Pool	13	29	5	✓	Database
23	gRPC BiDi Stream	5	9	5	✓	RPC
24	Blockchain Tx	6	8	6		Blockchain
25	Kafka Consumer	11	23	5	✓	Streaming
26	Rate Limiter	4	6	2		Resilience
27	Saga Orchestrator	20	37	20	✓	Distributed
28	Two-Phase Commit	7	8	7		Distributed
29	Leader Replication	13	24	3	✓	Distributed
30	Failover	6	8	3		Resilience

Table 8. Code metrics (all 7 phases).

Category	Files	Lines
Source (12 packages)	48	4,059
Test classes	19	6,297
Grand total	**67**	**10,356**

Table 9. Comparison of Java (Bica Reborn) and Python (Reticulate) implementations (full pipeline).

Aspect	Java	Python
AST nodes	7 records	7 dataclasses
Immutability enforcement	`List.copyOf`	`frozen=True`
Exhaustive dispatch	Sealed switch	`match`
Packages / modules	12	8
Source files	48	8
Source lines	~4,059	~1,800
Tests (total)	871	642
Benchmarks	30	30
Annotation processor	✓	—
CLI tool	✓	✓

Table 10. Package structure (12 packages, 48 source files).

Package	Key classes
`ast`	Sealed `SessionType` interface + 7 records
`parser`	`Tokenizer`, `Parser`, `PrettyPrinter`, `ParseError`
`statespace`	`StateSpace`, `StateSpaceBuilder`, `ProductStateSpace`
`lattice`	`LatticeResult`, `LatticeChecker`
`termination`	`TerminationResult`, `WFParallelResult`, `TerminationChecker`
`concurrency`	`ConcurrencyLevel`, `ConcurrencyLattice`, `MethodClassifier`, `ThreadSafetyChecker`
`annotation`	`@Session`, `@Shared`, `@ReadOnly`, `@Exclusive`
`morphism`	`MorphismKind`, `Morphism`, `GaloisConnection`, `MorphismChecker`
`processor`	`SourceMethodClassifier`, `SessionProcessor`
`typestate`	`TypestateChecker`, `TypestateResult`, `TypestateError`, `MethodCall`, `MethodCallExtractor`
`testgen`	`ViolationStyle`, `TestGenConfig`, `PathEnumerator`, `TestGenerator`
`cli`	`BicaCli`

implements the product construction $L(S_1) \times L(S_2)$ for the $\|$ constructor with componentwise transitions.

Phase 3: Concurrency, Annotations, and Annotation Processor (173 new tests, 461 total). `LatticeChecker` verifies that the state space forms a lattice using SCC-based quotient construction via iterative Tarjan's algorithm [11]. The concurrency lattice (Shared > {Sync, ReadOnly} > Exclusive) classifies methods for thread safety analysis. The annotation processor (`SessionProcessor`) performs the full pipeline at compile time: parse the `@Session` annotation, check termination and WF-Par, build the state space, verify lattice structure, and check thread safety of concurrent method pairs.

Phase 4: Morphism hierarchy (88 new tests, 549 total). `MorphismChecker` classifies relationships between state spaces into a hierarchy: isomorphism, order embedding, projection, and homomorphism. It also checks Galois connections (adjunction $\alpha(x) \leq y \iff x \leq \gamma(y)$) between state-space posets. `TerminationChecker` verifies that recursive definitions have exit paths and checks the six well-formedness conditions for $\|$ (WF-Par).

Phase 5: CLI tool (75 new tests, 624 total). `BicaCli` provides a standalone command-line interface. `BicaCli.run(args, out, err)` returns an integer exit code; `main()` calls `System.exit()`. The CLI supports text output (default), DOT output for Hasse diagram generation, and test generation output, with options for label suppression and custom titles.

Phase 6: Typestate checking (48 new tests, 781 total). `TypestateChecker` is a pure state-machine simulator that verifies method call sequences against session protocols. It collects all violations in a single pass rather than failing fast. `MethodCallExtractor` uses the `com.sun.source` API to extract method calls from source code at compile time, enabling the annotation processor to report violations as compiler diagnostics (§8).

Phase 7: Test generation (89 new tests, 871 total). `PathEnumerator` performs bounded DFS for valid paths, BFS for violation points, and prefix enumeration for incomplete protocols. `TestGenerator` renders the enumeration results as a compilable JUnit 5 source file. Across the 30 benchmarks, the generator produces 4,552 conformance tests (§9).

12 Related Work

Session types for objects. The original Bica tool [6] and its theoretical foundations [5, 12] established session types for object-oriented languages. Mungo [9, 10] is the most closely related tool: it type-checks Java objects against session type protocols specified in a companion `.protocol` file. Unlike Mungo, Bica Reborn uses inline annotations, supports the parallel constructor $\|$, and performs thread safety analysis on concurrent compositions.

Session type tools. Scribble [13] specifies multiparty session types as global protocols and projects them to local types. Bica Reborn operates at the object level (local types directly) and adds the lattice verification layer.

Lattice theory in program analysis. The use of lattices in static analysis originates with abstract interpretation [3]. Our contribution connects lattice structure to session types:

the parallel constructor introduces product lattices, making lattice properties (meets, joins, bounded completeness) directly relevant to protocol verification. The formal treatment of lattices follows Davey and Priestley [4].

Multiparty session types. Honda et al. [8] introduced multiparty session types with global protocols. The ‖ constructor in Bica Reborn is orthogonal: it models concurrent access to a *single* object by multiple threads, not multi-party communication between distributed endpoints.

13 Conclusion

We have presented Bica Reborn, a complete Java annotation-based session type checker for objects with parallel composition, implemented across seven phases. The implementation provides:

- An immutable, exhaustive AST using Java 21 sealed interfaces and records, with a tokenizer, recursive-descent parser, and round-trip–preserving pretty-printer supporting both ASCII and Unicode notation.
- State-space construction from session type ASTs, including product lattice construction for the ‖ constructor.
- Lattice verification via SCC-based quotient construction, ensuring all state spaces form bounded lattices with pairwise meets and joins.
- Termination checking and well-formedness analysis for parallel composition (WF-Par), covering termination, guardedness, variable scoping, and no nested ‖.
- A concurrency level lattice and thread safety analysis for concurrent method pairs in ‖ branches.
- A morphism hierarchy (isomorphism, embedding, projection, Galois connection) for comparing state-space structures.
- A Java annotation processor for compile-time verification of `@Session`, `@Shared`, `@ReadOnly`, and `@Exclusive` annotations.
- A standalone CLI tool for session type analysis.
- Typestate checking: a pure state-machine simulator that verifies method call sequences against session protocols at compile time.
- Automatic JUnit 5 test generation from session type state spaces, producing valid-path, violation, and incomplete-protocol tests (4,552 tests across the 30 benchmarks).

The implementation spans 12 packages (~4,059 source lines across 48 files), is validated by 871 unit tests across 19 test classes, and covers 30 benchmark protocols from 12 application domains. Both the Java and Python implementations achieve full pipeline parity and agree on all benchmark metrics.

Future work. Several directions remain for extending Bica Reborn:

1. **Nested parallelism:** support for $S_1 \parallel (S_2 \parallel S_3)$ and the associated n-ary product lattice construction.
2. **Subtyping with Sub-Par rules:** extend the subtyping relation to account for parallel composition, enabling safe protocol refinement.
3. **Bisimulation connection:** formally connect the lattice-based analysis to bisimulation equivalence on session types.
4. **Build tool integration:** Maven and Gradle plugins for seamless integration into standard Java build pipelines.
5. **IDE integration:** IntelliJ IDEA and Eclipse plugins providing real-time session type feedback during development.

The tool, reference implementation, and formal specification are available in the project repository.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

[2] Alexandre Zua Caldeira. 2025. *The Parallel Constructor ‖ for Session Types on Objects — BICA Reborn Formal Specification (Draft v0.2).* Technical Report. LASIGE, Faculdade de Ciências, Universidade de Lisboa. Internal document.

[3] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. *Conference Record of the Fourth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (1977), 238–252. https://doi.org/10.1145/512950.512973

[4] B. A. Davey and H. A. Priestley. 2002. *Introduction to Lattices and Order* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511809088

[5] Simon J. Gay and Malcolm Hole. 2005. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42, 2–3 (2005), 191–225. https://doi.org/10.1007/s00236-005-0177-z

[6] Simon J. Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. 2010. Modular Session Types for Distributed Object-Oriented Programming. In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL).* ACM, 299–312. https://doi.org/10.1145/1706299.1706335

[7] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *Lecture Notes in Computer Science*, Vol. 1381. 122–138. https://doi.org/10.1007/BFb0053567

[8] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2008. Multiparty Asynchronous Session Types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL).* ACM, 273–284. https://doi.org/10.1145/1328438.1328472

[9] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. 2016. Typechecking Protocols with Mungo and StMungo. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP).* ACM, 146–159. https://doi.org/10.1145/2967973.2968595

[10] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. 2017. Typechecking Protocols with Mungo and StMungo: A Session Type Toolchain for Java. In *Behavioural Types: from Theory to Tools.* River Publishers, 371–395.

[11] Robert Tarjan. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160. https://doi.org/10.1137/0201010

[12] Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. 2006. Type Checking a Multithreaded Functional Language with Session Types.

Theoretical Computer Science 368, 1–2 (2006), 64–87. https://doi.org/10.1016/j.tcs.2006.06.028

[13] Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. 2013. The Scribble Protocol Language. In *Trustworthy Global Computing (TGC)*. Springer, 22–41. https://doi.org/10.1007/978-3-319-05119-2_3